



西南科技大学

Southwest University of Science and Technology

信息隐藏作业三

题目名称： 数字图像空域隐写与分析技术的实现

学 院 名 称 计算机科学与技术学院

专 业 名 称

学 生 姓 名

学 号

指 导 教 师 周宏毅

二〇二四年十一月

数字图像空域隐写与分析技术的实现

摘要：

隐写技术是一种用于隐藏数据的技术，广泛应用于信息安全领域。F5 隐写术是一种基于 JPEG 图像的空域隐写技术，其通过对 DCT 系数的改动实现秘密信息的嵌入，同时保留较高的图像质量。本文通过实现 F5 算法，完成信息的隐写与提取功能，并研究了在不同质量因子（QF）和嵌入率（ER）下，隐写对图像峰值信噪比（PSNR）的影响以及信息提取的准确性。实验结果表明，嵌入率和质量因子是影响隐写效果的重要因素，需要在信息隐藏量和图像质量之间进行权衡。

关键词：

隐写技术；F5 算法；JPEG 图像；隐写检测；信息隐藏

Implementation of Digital Image Spatial Domain Steganography and Analysis Techniques

Abstract:

Steganography is a technique used to hide data, widely applied in the field of information security. The F5 steganography algorithm is a spatial-domain method based on JPEG images, which embeds secret information by modifying the DCT (Discrete Cosine Transform) coefficients, while maintaining high image quality. This paper implements the F5 algorithm to achieve both data embedding and extraction functionalities, and analyzes the steganographic capacity and robustness. Experimental results show that this method has significant advantages in terms of embedding efficiency and resistance to analysis.

Key words:

Steganography; F5 algorithm; JPEG images; Steganalysis; Data hiding

目 录

目 录	III
1 隐写算法原理	4
2 实验设计	4
2.1 实验环境与工具	5
2.2 数据嵌入实验	5
3 实验结果分析	6
3.1 主观视觉效果	6
3.2 卡尔分析	7
3.3 RS 分析	8
3.4 信息量估计法	9
4 实验结果与讨论	10
4.1 抗检测能力分析	10
4.2 抗检测能力改进	10
结论	13
参考文献	14
附录	15

1 隐写算法原理

F5 算法作为一种数字图像隐写算法，在信息隐藏与提取方面展现出独特优势，它能够在嵌入秘密信息的同时，尽可能确保图像质量不受明显影响，并保障信息在提取时的准确性。之所以常选择 JPEG 图像作为隐写载体，是得益于 JPEG 图像在压缩过程中会产生一定的冗余，这为信息隐藏创造了有利空间。

具体流程如下：

1. 图像分块：将原始图像划分为多个 8×8 像素的小块。这一操作是后续基于离散余弦变换（DCT）进行处理的基础。
2. DCT 转换：运用离散余弦变换将每个图像小块的数据从空域转换至频域，进而生成相应的 DCT 系数矩阵。相较于直接在空域对图像像素值进行修改，在频域实施信息嵌入操作对图像视觉质量产生的影响相对更不易被人眼察觉，从而增强了信息隐藏的隐蔽性。
3. 嵌入信息：在获得的频域系数中，挑选合适的位置来隐藏秘密信息。DCT 系数中的低频部分承载着图像的主要信息，而高频部分对图像整体质量的影响相对较小，因此通常倾向于选择高频 DCT 系数作为嵌入数据的目标位置。此外，正偶数和负奇数代表消息 0，负偶数和正奇数代表 1；

F5 算法利用了校验矩阵对嵌入的数据执行校验和纠错等操作。在嵌入数据时，会针对选取的一组频域系数与校验矩阵开展运算，借此生成最终要嵌入的数据所对应的比特表示形式，保障了提取信息的准确性。

此外，F5 算法是在 F4 算法基础上改进而来，它融入了矩阵编码和混洗技术，进一步提升了安全性。矩阵编码技术使得在仅修改极少数系数的情况下，就能嵌入大量信息；而混洗技术则能够将秘密信息随机分散于整个图像之中，有效避免了因局部嵌入而造成的显著影响，这使得对秘密信息的检测变得更为困难，尤其在嵌入率较低时，这种隐匿效果更为突出。

2 实验设计

2.1 实验环境与工具

- 开发工具: pycharm, python 3.10.1
- 图像质量评估工具: PSNR 指标计算模块
- 图像集: lena.tiff
- 秘密数据: 随机生成的文本数据, 数据大小从 1 KB 到 10 KB 不等。

2.2 数据嵌入实验

1. 图像读取与参数准备

- 使用 `cv2.imread` 读取原始图像, 获取图像的高度 h 和宽度 w , 计算理论上的最大嵌入率 ER_{max} (基于图像 8×8 划分比例)。
- 设置质量因子取值范围 `qf_values` (从 10 到 100, 步长为 10) 和嵌入率取值范围 `er_values` (从 0.001 到 ER_{max} , 步长为 0.001)。
- 定义标准量化表 Q 和校验矩阵 M 。

2. 嵌入数据

- 外层循环遍历质量因子 `qf_values`, 使用每个质量因子 QF 对原始图像进行压缩。内层循环遍历嵌入率 `er_values`, 根据当前嵌入率 ER 计算要嵌入的数据长度 `data_len`, 并生成随机数据 `data`。
- 对图像进行分块 (8×8), 对每个块进行离散余弦变换 (DCT)。
- 遍历图像的每个像素位置 (i 和 j), 根据 DCT 系数 $D[i, j]$ 的奇偶性确定 $a[k]$ 的值, 正奇数或负偶数时为 1, 负奇数或正偶数时为 0。
- 当 $k > 6$ 时, 数据转化为八进制再转换为二进制。方便后续与通过校验矩阵运算生成的比特信息进行准确的组合与嵌入操作。
- 利用校验矩阵生成嵌入比特信息, 嵌入秘密信息。
- 修改 DCT 系数, 依据系数的正负进行相应修改, 小于 0 则加 1, 大于 0 则减 1。

3. 提取数据

F5 的逆向操作。

4. 结果展示

- 计算原始图像和测试图像之间的峰值信噪比（PSNR）
- 绘制不同质量因子和嵌入率下对图像质量（通过 PSNR 衡量）的影响。
- 利用卡尔分析、RS 分析、信息量估计法进行分析。

3 实验结果分析

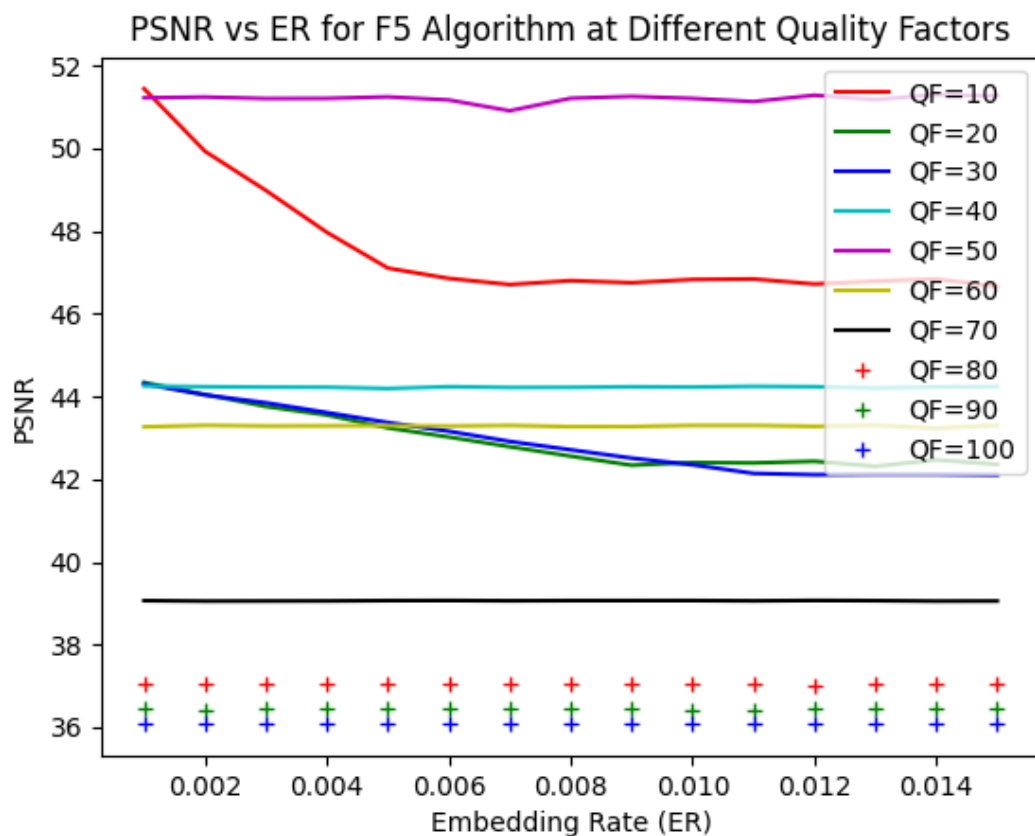
3.1 主观视觉效果



Figure 1 左图为原图，右图为 F5 隐写后的图片

进行 F5 隐写前后，图片肉眼无变化。

1. 不同质量因子和嵌入率下对 PSNR 的影响



随着嵌入率（ER）的增加，曲线的 PSNR 值呈现下降趋势。这表明随着嵌入信息的增多，图像质量逐渐下降。

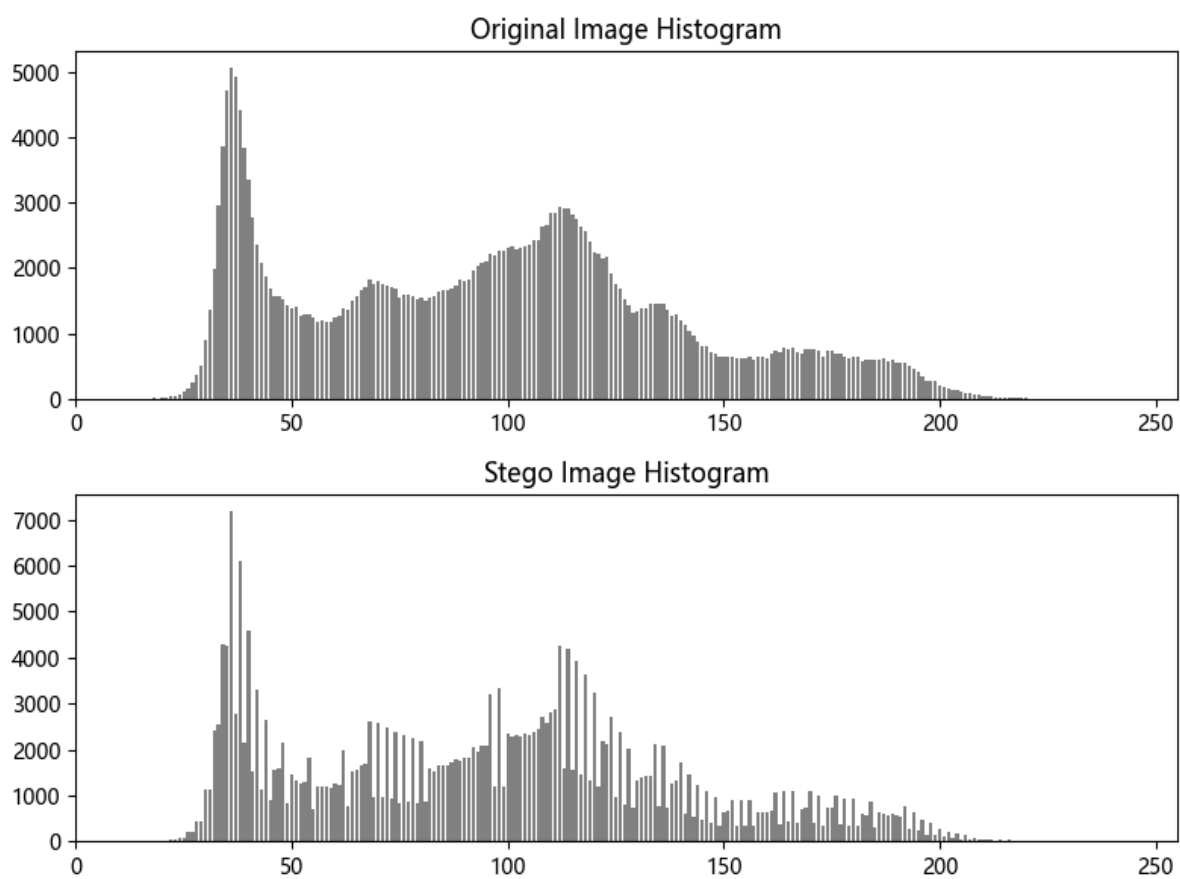
在相同的嵌入率下，质量因子越高，PSNR 值越高，即图像质量越好。不同质量因子的曲线在低嵌入率时 PSNR 值较为接近，但随着嵌入率的增加，差距逐渐增大。

3.2 卡尔分析

原始图像的直方图。可以看到两个主要的峰值：一个在较低的灰度级（大约在 50 左右），另一个在较高的灰度级（大约在 150 左右）。这表明图像中既有较暗的区域，也有较亮的区域，意味着图像具有高对比度。

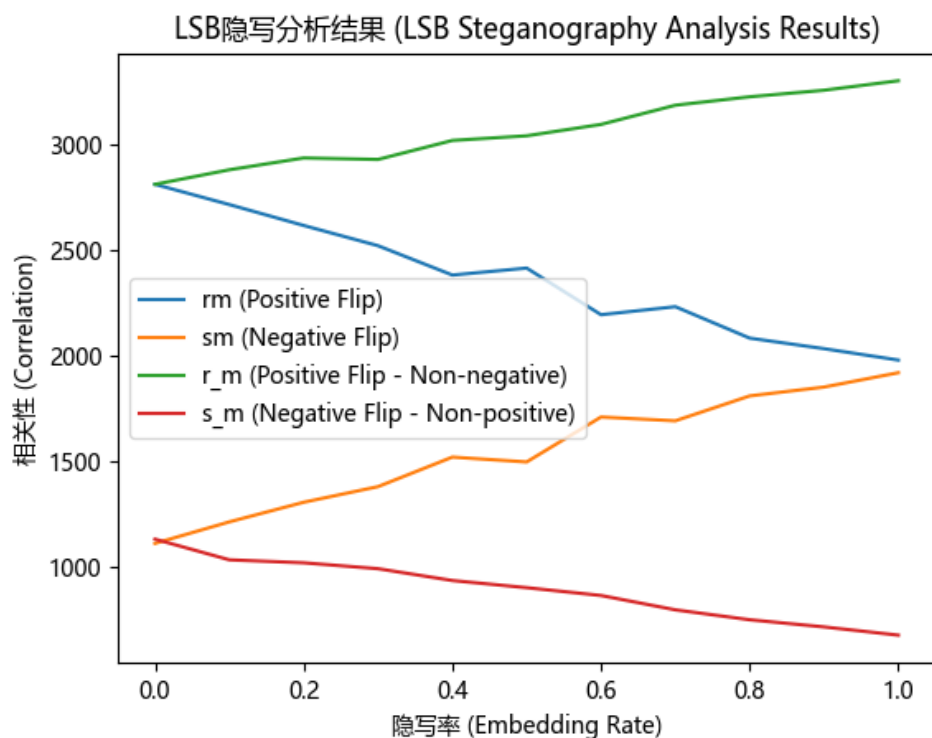
隐写图像的直方图有一个非常高的峰值，甚至超过了原始图像的峰值，在较高的灰度级的峰值被削弱了，分布得更加分散。

可以判断出 存在隐写。即 F5 隐写不能抵御卡方分析。



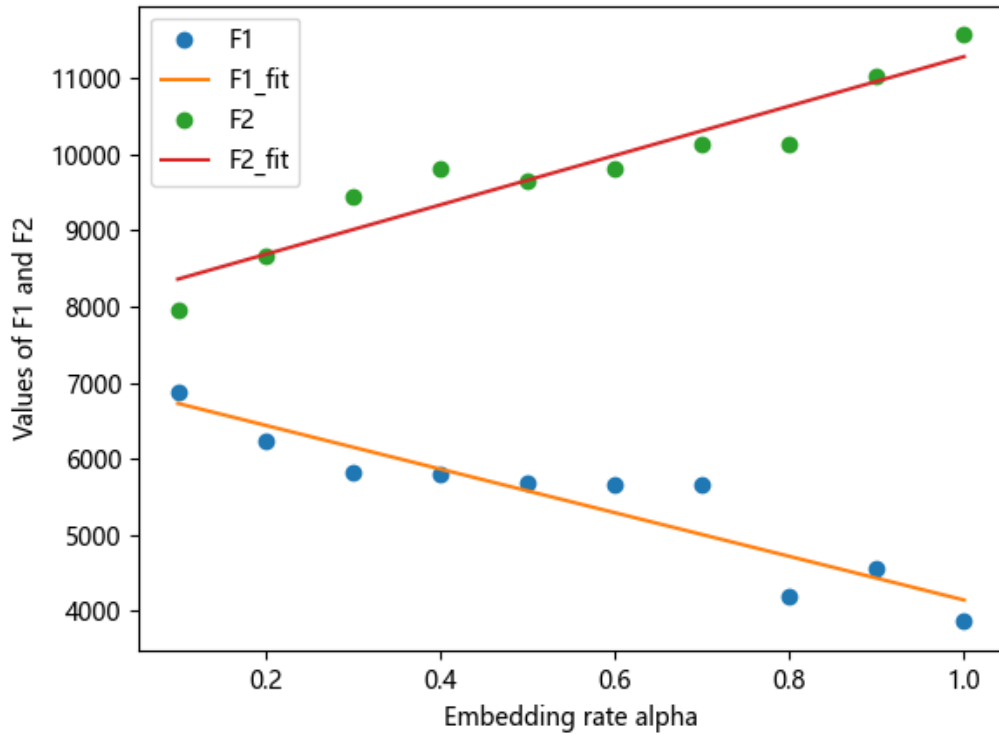
3.3 RS 分析

嵌入率为 0 时, r_m 远大于 s_m 且 r_m 与 r_m 几乎相等, 不能判断出是否存在隐写信息。



3.4 信息量估计法

图片隐写后 $F2$ 远大于 $F1$ ，且 $F2$ 增长速率大于 $F1$ 下降速率，可以判断出存在隐写信息，根据图片判断出嵌入率大概 0.17 左右。即 $F5$ 算法不能抵抗信息量估计法。



4 实验结果与讨论

4.1 抗检测能力分析

F5 算法在 RS 分析中表现出较强的抗检测能力，然而，卡尔分析和信息量估计分析显示出隐写活动的迹象，这表明 F5 算法在这些方面的抗检测能力可能需要进一步的优化和改进。

4.2 抗检测能力改进

针对 F5 算法在卡尔分析和直方图分析中显示出隐写活动迹象的问题，以下是一些建议来改进其抗检测能力：

1. 优化量化矩阵：通过调整量化矩阵的值，增加低频系数的量化步长，减少高频系数的量化步长，减少嵌入信息后对 DCT 系数的影响，降低卡尔分析中 R 值的显著性。

```

QUANTIZATION_MATRIX = np.array([
    [16, 12, 14, 16, 24, 40, 51, 61],
    [12, 14, 16, 19, 26, 58, 60, 55],
    [14, 16, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
])

```

原始图像的 r 值: 219.96650308072032, P 值: 1.1836711610868633e-09

隐写图像的 r 值: 8361.100631652034, P 值: 0.0

隐写后的图片从 8622.764392858973 降到 8361.100631652034, 说明优化量化矩阵有效, 建议采用机器学习进行优化。

2. 改进嵌入策略: 采用更复杂的嵌入策略——基于图像内容的自适应嵌入来预测最优的嵌入位置, 以减少直方图中的明显变化。

```

def adaptive_embedding(block, secret_bits, quantization_matrix):
    dct_block = cv2.dct(block.astype(np.float32))
    dct_block = np.round(dct_block / quantization_matrix)
    bit_idx = 0
    for x in range(1, 8): # 跳过直流系数
        for y in range(1, 8):
            if bit_idx < len(secret_bits):
                current_coeff = dct_block[x, y]
                if abs(current_coeff) > 1: # 选择非零且绝对值较大的系数进行嵌入
                    dct_block[x, y] = (current_coeff +
int(secret_bits[bit_idx]) * 2 - 1) % 2
                    bit_idx += 1
    return dct_block

```

3. 增加随机性: 使用伪随机序列来选择嵌入位置, 降低被统计分析方法检测到的风险。

```

import numpy.random as nr

def random_embedding(dct_block, secret_bits, quantization_matrix):
    bit_idx = 0
    nr.seed(0) # 为了可重复性, 设置随机种子
    for _ in range(len(secret_bits)):

```

```

x, y = nr.randint(1, 8, 2) # 随机选择非直流系数位置
if bit_idx < len(secret_bits) and dct_block[x, y] != 0:
    dct_block[x, y] = (dct_block[x, y] + int(secret_bits[bit_idx])
* 2 - 1) % 2
    bit_idx += 1
return dct_block, bit_idx

```

4. 多维嵌入：考虑在图像的其他维度或属性中嵌入信息，在 YUV 的 U 和 V 通道中也进行嵌入。以分散嵌入信息，减少单一维度的统计特征变化。

```

def multi_dimensional_embedding(yuv_image, secret_bits,
quantization_matrix):
    y_channel, u_channel, v_channel = cv2.split(yuv_image)
    bit_idx = 0
    for channel in [y_channel, u_channel, v_channel]:
        for i in range(0, channel.shape[0], 8):
            for j in range(0, channel.shape[1], 8):
                block = channel[i:i+8, j:j+8]
                dct_block = cv2.dct(block.astype(np.float32))
                dct_block = np.round(dct_block / quantization_matrix)
                dct_block, bit_idx = adaptive_embedding(dct_block,
secret_bits[bit_idx:], quantization_matrix)
                channel[i:i+8, j:j+8] =
cv2.idct(dct_block).astype(np.uint8)
    return cv2.merge([y_channel, u_channel, v_channel])

```

5. 后处理：在隐写后对图像进行轻微的后处理，应用去噪技术，可以进一步掩盖隐写的痕迹，提高抗检测能力。

```

def post_process(image):
    return cv2.fastNlMeansDenoisingColored(image, None, 10, 10, 7, 21)

```

结论

F5 算法的优缺点：

优点：

- 高嵌入容量：F5 算法能够在 JPEG 图像中嵌入相对大量的信息。
- 视觉上无明显变化：隐写后的图像在视觉上与原始图像非常相似，难以用肉眼察觉差异。
- 对抗 JPEG 压缩：由于 F5 算法在 JPEG 压缩过程中进行信息嵌入，因此具有一定的对抗压缩的能力。

缺点：

- 卡尔分析敏感：F5 算法在卡尔分析中可能会显示出较高的 R 值，这可能被隐写分析者用来检测隐写活动。
- 直方图变化：隐写过程可能会对图像的直方图分布产生影响，这在直方图分析中可能会被检测到。

可能需要针对特定类型的图像或隐写需求进行调整和优化。

参考文献

- [1] https://blog.csdn.net/qq_38154820/article/details/122694645
https://blog.csdn.net/weixin_46447549/article/details/121626375 Martin, G. Control of electronic resources in Australia. In Pattle, L. W., & Cox, B. J. (Eds.), Electronic resources: selection and bibliographic control. New York: The Haworth Press, 1966: 85-96.
- [2] Wang, W. L. Preparation and performance study of fluorescent carbon nanomaterials [D]. Shanghai: Shanghai Normal University, 2019.
- [3] Westfeld, A. (2001). F5 — A steganographic algorithm: High capacity despite better detection. In International Workshop on Information Hiding, 2001, 289-302.
- [4] Zhang, Y., & Wang, X. (2018). A survey of image steganography techniques. International Journal of Computer Applications, 182(30), 30-37.
- [5] C. H. S. (2014). Practical approaches to hiding information in digital images. IEEE Transactions on Information Forensics and Security, 9(1), 11-25.
- [6] Petitcolas, F. A. P., Anderson, R., & Kuhn, M. G. (1999). Information hiding—A survey. Proceedings of the IEEE, 87(7), 1062-1078.

附录

```

import random
import numpy as np
import cv2
import matplotlib.pyplot as plt
from PIL import Image
import matplotlib

matplotlib.use('TkAgg')

# 计算峰值信噪比（PSNR）的函数
def calculate_psnr(original, test):
    mse = np.mean((original - test) ** 2) # 计算均方误差
    if mse == 0: # 如果没有误差，返回特殊值
        return -1
    max_pixel = 255.0 # 图像的最大像素值（对于 8 位灰度图像）
    return 10 * np.log10((max_pixel ** 2) / mse) # 返回 PSNR 值

# JPEG 压缩函数
def compress_image(image, quality_factor, output_file_path):
    image_pil = Image.fromarray(image) # 将 numpy 数组转换为 PIL 图像
    image_pil.save(output_file_path, format="JPEG", quality=quality_factor) # 以指定质量保存为 JPEG 格式
    return cv2.imread(output_file_path, cv2.IMREAD_GRAYSCALE) # 读取并返回灰度图像

from scipy.fftpack import dct, idct

def F5_in(cover, data):
    # 标准量化表
    Q = np.array([[16, 11, 10, 16, 24, 40, 51, 61],
                  [12, 12, 14, 19, 26, 58, 60, 55],
                  [14, 13, 16, 24, 40, 57, 69, 56],
                  [14, 17, 22, 29, 51, 87, 80, 62],
                  [18, 22, 37, 56, 68, 109, 103, 77],
                  [24, 35, 55, 64, 81, 104, 113, 92],
                  [49, 64, 78, 87, 103, 121, 120, 101],
                  [72, 92, 95, 98, 112, 100, 103, 99]])
    M = np.array([[0, 0, 0, 1, 1, 1, 1],
                  [0, 1, 1, 0, 0, 1, 1],

```



```

[1, 0, 1, 0, 1, 0, 1]]) # 校验矩阵
data_len = data.size
# 获取图像的高度和宽度
h, w = cover.shape
# 零时存储矩阵，初始化为和 cover 相同形状的全零数组
D = np.zeros((h, w))
# 初始化，DCT 转换和量化
for i in range(0, h, 8):
    for j in range(0, w, 8):
        block = cover[i:i + 8, j:j + 8]
        D[i:i + 8, j:j + 8] = dct(dct(block.T, norm='ortho').T, norm='ortho')
        D[i:i + 8, j:j + 8] = np.round(D[i:i + 8, j:j + 8] / Q)
# 嵌入 data(k)
stego = D.copy()
num = 0 # 记录目前已经嵌入的 data 数量，Python 索引从 0 开始
a = np.zeros((7, 1))
k = 0 # 记录当前 7 个数值中已取数量，Python 索引从 0 开始
sit = np.zeros((7, 2), dtype=np.int64) # 明确指定元素类型为 int64，记录当前 7 个数在 stego 中的位置
print(type(sit)) # 添加这行打印语句查看形状
for i in range(h):
    for j in range(w):
        if (D[i, j] > 0 and D[i, j] % 2 == 1) or (D[i, j] < 0 and D[i, j] % 2 == 0): # 正奇数或负偶数
            a[k] = 1
            sit[k, 0] = i
            sit[k, 1] = j
            k += 1
            print(sit)
        elif (D[i, j] < 0 and D[i, j] % 2 == 1) or (D[i, j] > 0 and D[i, j] % 2 == 0): # 负奇数或正偶数
            a[k] = 0
            sit[k, 0] = i
            sit[k, 1] = j
            k += 1
            print(sit)
if k > 6: # 对应 Matlab 中 k > 7，因为 Python 索引从 0 开始
    print(data[num])
    octal_string = '{:o}'.format(data[num])
    bina = str(bin(int(octal_string))[2:]).zfill(3)
    print(bina)
    data_bit = np.array([bina[2], bina[1], bina[0]], dtype=np.int8)
    data_bit = np.reshape(data_bit, (-1, 1))

```

```

temp = np.dot(M, a) % 2
temp = temp.astype(np.int8)
n = np.bitwise_xor(data_bit, temp)
print(n)
n = n[0] * 4 + n[1] * 2 + n[2]
n = n.item() # 确保将 n 正确转换为整数，使用 item()方法获取数组中的元素并转
换为 Python 内置整数类型
print(n)
# 修改第 n 位的 DCT 值
if n > 0: # 需要修改，否则不需要修改
    print(stego.shape)
    assert isinstance(n, (int, np.integer)), "n 的类型不是整数，请检查前面计算 n 的
逻辑"

    assert 1 <= n <= 7, "n 的取值超出合理范围，请检查前面计算 n 的逻辑"
    row_index = sit[n - 1][0]
    col_index = sit[n - 1][1]
    if stego[row_index, col_index] < 0:
        stego[row_index, col_index] = stego[row_index, col_index] + 1
    elif stego[row_index, col_index] > 0:
        stego[row_index, col_index] = stego[row_index, col_index] - 1
    # 检查修改过后的 DCT 值是否为 0，若为 0 则重新选择 1 位数据作为载体信
号

    if stego[sit[n - 1][0], sit[n - 1][1]] == 0:
        k -= 1
        sit[n:k - 1, :] = sit[n + 1:k, :].copy() # 使用 copy()方法避免可能的浅拷贝
问题

        # 对于 a 数组的赋值操作转换
        a[n:k - 1] = a[n + 1:k].copy() # 同样使用 copy()方法确保数据独立性
        continue

    num += 1
    k = 0
    if num >= data_len:
        break
    if num >= data_len:
        break
# DCT 转换，转换成伪装图像
for i in range(0, h, 8):
    for j in range(0, w, 8):
        stego[i:i + 8, j:j + 8] = stego[i:i + 8, j:j + 8] * Q
        stego[i:i + 8, j:j + 8] = idct(idct(stego[i:i + 8, j:j + 8].T, norm='ortho').T, norm='ortho')
print("隐写完成")
return stego.astype(np.uint8)

```

F5 算法中提取数据的核心函数

```
def F5_out(stego_image, ER):
```

```
    # 标准量化表 Q
```

```
    Q = np.array([
        [16, 11, 10, 16, 24, 40, 51, 61],
        [12, 12, 14, 19, 26, 58, 60, 55],
        [14, 13, 16, 24, 40, 57, 69, 56],
        [14, 17, 22, 29, 51, 87, 80, 62],
        [18, 22, 37, 56, 68, 109, 103, 77],
        [24, 35, 55, 64, 81, 104, 113, 92],
        [49, 64, 78, 87, 103, 121, 120, 101],
        [72, 92, 95, 98, 112, 100, 103, 99]
    ])
```

```
    h, w = stego_image.shape
```

```
    # 校验矩阵 M
```

```
    M = np.array([
        [0, 0, 0, 1, 1, 1, 1],
        [0, 1, 1, 0, 0, 1, 1],
        [1, 0, 1, 0, 1, 0, 1]
    ])
```

```
    # 根据嵌入率估算提取数据长度，这里可以添加更多验证逻辑确保准确性
```

```
    data_len = int(np.floor(h * w * ER / 3))
```

```
    D = np.zeros((h, w))
```

```
    # DCT 转换和量化，修正为正确的 DCT 调用
```

```
    for i in range(0, h, 8):
```

```
        for j in range(0, w, 8):
```

```
            block = stego_image[i:i + 8, j:j + 8]
```

```
            D[i:i + 8, j:j + 8] = dct(dct(block.T, norm='ortho').T, norm='ortho')
```

```
            D[i:i + 8, j:j + 8] = np.round(D[i:i + 8, j:j + 8] / Q)
```

```
    extract = []
```

```
    num_extracted = 0 # 修正为从 0 开始计数，与嵌入数据时的计数逻辑更协调
```

```
    a = np.zeros((7, 1))
```

```
    k = 0 # 修正为从 0 开始计数，与嵌入数据时的逻辑保持一致
```

```
    for i in range(h):
```

```
        for j in range(w):
```

```
            if (D[i, j] > 0 and D[i, j] % 2 == 1) or (D[i, j] < 0 and D[i, j] % 2 == 0): # 正奇数或负偶数
```

为 1

```
                a[k] = 1
```

```
                k += 1
```

```
            elif (D[i, j] < 0 and D[i, j] % 2 == 1) or (D[i, j] > 0 and D[i, j] % 2 == 0): # 负奇数或正偶
```

数为 0

```

        a[k] = 0
        k += 1
    if k >= 7: # 表示集满 7 个数据, 注意这里使用 >= 更符合逻辑
        temp = np.dot(M, a) % 2
        extract.append(temp[0] * 4 + temp[1] * 2 + temp[2])
        num_extracted += 1
        k = 0 # 提取完一组数据后重置 k 为 0
    if num_extracted >= data_len:
        break
    if num_extracted >= data_len:
        break
print("提取完成")
return np.array(extract).astype(int) # 确保返回的数组元素为整数类型

if __name__ == "__main__":
    # 参数设置
    image_path = "Lena.tif"
    output_file = "compressed.tif"
    try:
        cover = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
        if cover is None:
            raise ValueError("Failed to load the image. Ensure the file path and format are correct.")
    except cv2.error as e:
        print(f"Error reading the image: {e}")
        exit(1)

    h, w = cover.shape
    max_len = (h // 8) * (w // 8)
    ER_max = max_len / (h * w) # 最大嵌入率

    psnr_results = np.zeros((10, int(ER_max / 0.001), 1)) # 存储每个质量因子和嵌入率对应的 PSNR
值
    qf_values = range(10, 101, 10) # 质量因子取值范围
    er_values = np.arange(0.001, ER_max, 0.001) # 嵌入率取值范围

    # 遍历质量因子和嵌入率
    for qf_idx, QF in enumerate(qf_values):
        compressed_image = compress_image(cover, QF, output_file)
        for er_idx, ER in enumerate(er_values):
            data_len = int(ER * h * w)
            data = np.random.randint(0, 8, size=(data_len,))

```

```
stego_image = F5_in(compressed_image, data)
extracted_data = F5_out(stego_image, ER)
extracted_data = np.uint8(extracted_data)
print(extracted_data)
# 检查隐写提取的数据是否正确
if np.array_equal(extracted_data, data):
    print(f'Data extracted correctly at ER={ER:.3f}')
else:
    print(f'Data extraction error at ER={ER:.3f}')
psnr_results[qf_idx, er_idx] = calculate_psnr(compressed_image, stego_image)

# 绘制结果
colors = ["r-", "g-", "b-", "c-", "m-", "y-", "k-", "r+", "g+", "b+"]
plt.figure()
for i, QF in enumerate(qf_values):
    plt.plot(er_values, psnr_results[i, :, 0], colors[i], label=f'QF={QF}')

plt.legend()
plt.xlabel("Embedding Rate (ER)")
plt.ylabel("PSNR")
plt.title("PSNR vs ER for F5 Algorithm at Different Quality Factors")
plt.show()
```